

Memory Management in Macaulay2

The Garbage Collector Today,
and a Plan to Confine It

Prepared for the Macaulay2 engine group

Andrew Tawfeek, with Claude (Fable 5)

July 21, 2026 · draft for discussion

Chapter 1 describes, in plain language but with full technical detail, how memory is actually managed in Macaulay2 today. Chapter 2 is an architectural plan for confining the garbage collector to the front end, addressing the requests Mike Stillman made at the July 2026 engine meeting.

About this document

This document does two jobs. **Chapter 1** explains how memory is actually managed in Macaulay2 today — who allocates what, where the garbage collector is involved, where it is deliberately avoided, and what it costs. **Chapter 2** is an architectural plan for the change Mike Stillman asked for at the July 2026 engine meeting: keep the collector for the front end, remove it from the computational core of the engine (“the garbage collector . . . should be set up only for things in the interface directory”). The agreed deliverable was a proposal the group edits *before* anyone writes a large pull request — detailed enough for humans to evaluate and for supervised AI agents to later execute step by step. It replaces the earlier two-page summary ([src/gc-current-state.pdf](#)), which was accurate but too dense to actually read.

Sources, and how much to trust this

Everything here comes from two places: the **meeting transcript** (Mike’s requests, with timestamps and quotes, are collected in [src/mike-requests.md](#), and Chapter 2 quotes them inline so the plan can be checked against what was asked), and the **source tree itself**, read as of July 21, 2026. File paths are cited throughout (relative to the repository root); every factual claim about the code was verified against the tree rather than recalled from memory. Line numbers are avoided on purpose — they rot. The draft was produced by Andrew working with Claude (Fable 5), per the plan agreed at the meeting; it is a *proposal*, meant to be marked up, reordered, or vetoed — especially by Mike, who knows where the bodies are buried.

How to read it

Colored boxes carry the structure: **In brief** boxes open each chapter and major section (read only these and you still get the correct picture); **In plain terms** boxes translate mechanisms into ordinary language; **What Mike asked for** boxes quote the meeting request a section answers; **A cautionary tale** records history that must not repeat. If you know the engine well, skim Chapter 1 by its boxes, then read Chapter 2 closely; newcomers can read straight through. A glossary is in Appendix A.

Contents

About this document	1
1 How Macaulay2 Manages Memory Today	3
1.1 Why Macaulay2 has a garbage collector at all	3
1.2 The Boehm collector, explained honestly	4
1.3 How the collector is wired into Macaulay2	5
1.4 The life of a value: three walkthroughs	8
1.5 A census: who allocates where	10
1.6 The border crossings today	13
1.7 What the current arrangement costs	14
1.8 The seam is already half-built	16
2 A Plan for Confining the Collector	17
2.1 What Mike asked for	17
2.2 The target architecture: three zones and one rule	18
2.3 Ground rules for the whole migration	19
2.4 Workstream 1: harden the border	20
2.5 Workstream 2: the matrix split	21
2.6 Workstream 3: ARings all the way out	22
2.7 Workstream 4: FLINT by default	23
2.8 Workstream 5 (adjacent): cache-friendly polynomials	24
2.9 The roadmap: phases, acceptance, executors	24
2.10 Risk register	25
2.11 What this plan deliberately does not do	25
2.12 Open questions for the group	27
2.13 The next two weeks	27
A Glossary	28

Chapter 1

How Macaulay2 Manages Memory Today

In brief

Macaulay2 runs on top of a single garbage collector — the Boehm–Demers–Weiser conservative collector — which manages nearly every object in the system: every interpreter value the user creates, and most C++ objects inside the engine. This was a deliberate bargain struck at the start of the project: one collector for everything makes the boundary between interpreter and engine trivially simple, at the price of collector overhead inside computations. The engine already avoids the collector in its hottest arithmetic (the ARing element types, external libraries, and some arenas), and it bridges back into collected memory at well-defined crossing points. This chapter walks through the whole arrangement: the collector itself (§1.2), how it is wired into the system (§1.3), the life of a value from keyboard to engine and back (§1.4), a census of who allocates where (§1.5), the existing border crossings (§1.6), and what all of it costs (§1.7).

1.1 Why Macaulay2 has a garbage collector at all

Start with the problem the collector solves, because it solves it well.

A Macaulay2 session is an open-ended conversation. The user makes a ring, then a matrix over it, then an ideal, then a resolution; the resolution quietly holds onto the matrix, the matrix holds onto the ring, and an hour later some of these objects matter and some are long forgotten. Nobody — least of all the user — can say at any given moment which objects are still needed. Lifetimes at top level are genuinely unknowable in advance, they form an arbitrary tangled graph, and the whole point of an interactive system is that the user never thinks about this. This is precisely the situation garbage collection was invented for, and Mike reaffirmed the choice at the July meeting: “the garbage collector is great for user objects ... they just go away when you don’t need them anymore.”

The front end is committed to this design at the language level. The interpreter is written in Macaulay2’s own *D language* and translated to C; D has **new** but no **free**, so every string, list, hash table, and closure the interpreter ever creates is handed to the collector. There is no plausible version of the front end without a garbage collector, and nobody is proposing one.

The historical bargain concerns the *engine*. When the C++ engine was built, it adopted the same collector for its own objects — rings, matrices, polynomials, computations. This bought one enormous simplification: an engine object handed to the interpreter needs no destructor protocol, no reference counting, no ownership negotiation at the boundary. If the user’s session can still reach the object (through any chain of interpreter values), the collector keeps it; when the session forgets it, the collector reclaims it. One collector, one heap, one rule.

The price of the bargain also lives in the engine. A Gröbner basis computation allocates furiously in tight loops, and those allocations have none of the hard-lifetime character of top-level objects: inside a computation, lifetimes are structured, scoped, and knowable — exactly the situation where C++ manages memory beautifully on its own. Running such code on a conservative collector means paying for heap scans the computation never needed. Mike’s estimate of the damage has grown over the years: “I used to think it was about a 20% hit ... I would say sometimes it’s a multiple-times hit; it spends too much time in garbage collection.” And the empirical converse: “whenever we can get rid of the garbage collector inside of any computation type, then it speeds things way up.”

That asymmetry is the thesis of this whole document, so it deserves one sentence in italics: *lifetimes are hard at top level and easy inside a computation — so collect the top level, and let C++ own the inside*. Chapter 2 is the plan for finishing that separation. The rest of this chapter establishes where the line runs today.

1.2 The Boehm collector, explained honestly

Macaulay2 uses *bdwgc*, the Boehm–Demers–Weiser collector — a mature, widely used C library (it also sits under Guile, GNU Java, and many other systems). Understanding four of its properties explains almost everything about how Macaulay2 behaves.

It is a drop-in replacement for `malloc`. Code allocates with `GC_MALLOC` instead of `malloc` and simply never frees. From the programmer’s point of view memory management disappears. Periodically — triggered by allocation volume — the collector pauses the program, figures out which blocks can still be reached, and recycles the rest.

It is conservative: it does not know what your data means. A precise collector (Java’s, OCaml’s) knows the layout of every object — which words are pointers and which are integers. The Boehm collector knows nothing. It finds live memory by scanning *roots* — every thread’s stack and registers, global variables, and explicitly registered blocks — and treating every word whose bit pattern *could* be an address in its heap as if it were a pointer. Everything so reachable is *marked*; every unmarked block is *swept* back into the free lists. This is classic mark-and-sweep, done by guesswork.

In plain terms

The collector is a cleaning crew that cannot read the hotel’s guest register. It walks every corridor and keeps any room that *looks* occupied. Two mistakes are possible. It may keep a room forever because a number on a whiteboard happens to look like that room’s number (*false retention* — wasteful, but safe). Far worse, if a guest writes their room number in code the crew doesn’t recognize, the room looks vacant and gets cleared out while the guest is still living there (*disguised pointers* — a crash). The first mistake costs memory; the second is the reason FLINT integers cannot live in this hotel, as §1.7.4 explains.

Conservatism has three consequences that matter to us:

1. **False retention.** As Mike put it at the meeting: “if there’s an integer in there that looks like a pointer and it follows it, it could get garbage” — a coincidental bit pattern keeps a dead object (and everything it references) alive. The practical mitigation is `GC_MALLOC_ATOMIC`: when the caller promises a block contains no pointers (strings, arrays of number data), the collector skips scanning it, which both speeds marking and removes a source of coincidences. Macaulay2 uses atomic allocation systematically.
2. **No disguised pointers.** The collector recognizes addresses, not intentions. A pointer that has been tagged, shifted, or otherwise encoded — as FLINT encodes its small-integer optimization — is invisible to the mark phase, and the memory it protects will be reclaimed out from under the program. Any data structure that plays pointer games is therefore banned from the collected heap.
3. **Objects never move.** Since any “pointer” might really be an integer, the collector cannot rewrite references, so it can never compact the heap. Long sessions fragment; large matrices must find contiguous holes.

It stops the world. During marking, every registered thread is halted. The pause is proportional to the amount of *scannable* live memory — which is why a computation that fills the heap with pointer-bearing nodes (say, millions of linked-list polynomial terms, §1.5) pays twice: once to allocate, and again every time the collector re-walks those nodes. `bdwgc` offers incremental and generational modes that shorten pauses, but the scanning work itself does not go away.

It supports finalizers. Code can register a callback to run when a particular block becomes unreachable. This is the sanctioned escape hatch for tying non-collected resources (a `malloc`’d FLINT matrix, an open file) to the lifetime of a collected object, and the engine leans on it in exactly this way (§1.6). Finalizers come with well-known subtleties — they run at unpredictable times, on unpredictable threads, and reentrancy is easy to get wrong; §1.7.4 recounts a bug in this area that cost Mike weeks.

1.3 How the collector is wired into Macaulay2

In brief

One header (`include/M2/gc-include.h`) configures the collector for the whole system; `bin/main.cpp` starts it and pre-grows the heap; a single allocation funnel (`getmem/getmem_atomic` in `e/interface/m2-mem.cpp`) feeds both halves of the system. The front end’s compiler emits a GC allocation for every `new`; engine classes opt in by inheriting `our_new_delete`. Freeing, outside of two deliberate exceptions, is literally commented out.

1.3.1 Configuration and startup

Everything begins in `include/M2/gc-include.h`, the one header through which all of Macaulay2 sees the collector. It defines `GC_THREADS` before including `gc/gc.h` — which causes `bdwgc`’s own headers to redirect `pthread_create` to `GC_pthread_create`, so every thread the system ever spawns is automatically registered with the collector — and it bakes in a compile-time initial heap of 70 MB (`GC_INITIAL_HEAP_SIZE 70000000`).

At startup, `Macaulay2/bin/main.cpp` initializes the collector and immediately raises that floor:

```
GC_INIT();
size_t heap_size = GC_get_heap_size(),
      min_heap_size = 150'000'000 + getMaxAllowableThreads() * 8'000'000;
if (heap_size < min_heap_size)
    GC_expand_hp(min_heap_size - heap_size);
```

so a real session starts with roughly 190–290 MB of collected heap depending on core count. The pre-growth is a tuning act: a bigger heap means fewer collections. Users can push further with environment variables (`GC_INITIAL_HEAP_SIZE=20G` M2 and friends, documented in the “GC garbage collector” node of the Macaulay2 documentation, `packages/Macaulay2Doc/ov_prelude.m2`), and the language exposes `collectGarbage` (a direct call to `GC_gccollect`) and `GCstats` for inspection. Notably, that is the *entire* runtime tuning surface: Macaulay2 never enables bdwgc’s incremental mode or adjusts interior-pointer handling — it runs the collector in its default stop-the-world configuration, with parallel marking enabled in the build. By default the build compiles bdwgc itself (version 8.2.12, pinned as the `submodules/bdwgc` submodule, configured for large heaps); `-with-system-gc` substitutes a system copy.

1.3.2 One funnel: `getmem`

Both halves of the system allocate through one pair of functions in `e/interface/m2-mem.cpp`: `getmem`, which wraps `GC_MALLOC` (scanned memory, for data that may contain pointers), and `getmem_atomic`, which wraps `GC_MALLOC_ATOMIC` (unscanned memory, for strings and number data — the “atomic” promise of §1.2). The matching `freemem` is the most honest few lines in the codebase:

```
void freemem(void *s) {
    TRAPCHK(s);
#ifdef MEMDEBUG
    M2_debug_free(s);
#else
    /* GC_FREE(s); */
#endif
}
```

The free is *commented out*. In a normal build, “freeing” memory in Macaulay2 does nothing at all; reclamation belongs entirely to the collector. Only two places in the whole tree still truly release GC memory: the SCC compiler’s internal scope stack, and the destructor of the engine’s old `stash` allocator (`e/mem.cpp`, annotated “this dramatically improves our memory usage”).

1.3.3 How the front end allocates: the compiler does it

The interpreter is written in the D language and translated to C by SCC (`Macaulay2/c/`). The translator emits the allocation call for every `new` expression itself — the choice between scanned and atomic memory is made per type, at compile time, in one line of the code generator (`c/cprint.c`):

```
put(pointer_to_atomic_memory(t) ? "GC_MALLOC_ATOMIC" : "GC_MALLOC");
```

A type whose fields are all pointer-free (a string, an array of ints) is allocated atomically; anything that might hold a pointer gets scanned memory. The consequence is worth stating plainly: *every value the interpreter ever creates — every list, string, hash table, closure, and big integer at top*

level — is a garbage-collected object, by construction. There is no other allocator in the front end, and this document proposes no change to that.

1.3.4 How the engine allocates: classes opt in

The C++ engine joins the collected heap through inheritance. `e/newdelete.hpp` defines a base class `our_new_delete` whose `operator new` routes through `getmem`:

```
static inline void *operator new(size_t size)
{ void *p = getmem(size); if (p == NULL) outofmem2(size); return p; }
```

Any class deriving from it — 57 files’ worth of classes across the engine — has all its instances allocated in scanned GC memory. On top of it, `e/hash.hpp` defines the two roots of the engine’s object hierarchy:

- `EngineObject` : `our_new_delete` — immutable objects with a lazily computed hash: `RingElement`, `Matrix`, `FreeModule`, `MonomialIdeal`, ...
- `MutableEngineObject` : `our_gc_cleanup` — mutable objects: `Ring` and all its subclasses, `MutableMatrix`, and every `Computation` (Gröbner bases, resolutions, ...). `our_gc_cleanup` is the GC-destructor bridge: its constructor registers a finalizer that runs the object’s virtual destructor when the collector reclaims it (via `GC_REGISTER_FINALIZER_IGNORE_SELF`).

The same header supplies the engine’s collected containers and array macros: `gc_vector<T>` (a `std::vector` over `gc_allocator`, used for instance for a `Matrix`’s column array), and `newarray/newarray_atomic` for raw GC arrays. The thread supervisor uses the analogous `gc_map/gc_set` (`system/gc_std.hpp`).

Two details of the opt-in machinery deserve a note because Chapter 2 touches them:

- **The vestigial `stash`.** The engine still carries its pre-GC free-list allocator (`e/mem.hpp`); the default Gröbner engine, the Hilbert-series code, and the Schreyer-resolution monomial tables still construct `stash` pools. But the pooling was short-circuited long ago: `stash::new_elem()` now just calls the GC (its first line returns a fresh `newarray_clear`), so these “pools” provide statistics and nothing else. They are fossils of exactly the owned-allocator pattern Chapter 2 reintroduces deliberately.
- **A live TODO in the tree agrees with this document.** The free-module vector node — the linked-list cell from which matrices are built — reads, verbatim (`e/rings/ringelem.hpp`):

```
typedef struct vecterm *vec;
// TODO: why is this garbage collected?
struct vecterm : public our_new_delete { vec next; int comp; ring_elem coeff; };
```

1.3.5 One historical note: GMP used to live in the collected heap

Nothing in the tree calls `mp_set_memory_functions` — GMP, MPFR, FLINT, Factory, and math-icgb all allocate with plain `malloc`, outside the collector’s world. That was not always so. A comment in `cmake/build-libraries.cmake` preserves the history:

“mpfr puts pointers to gmp numbers in thread local variables, unless specially configured, so we shouldn’t tell gmp to use libgc (we used to do that).”

Macaulay2 once pointed GMP’s allocator at the collector, so that every big integer was born collected; it was undone because library-internal pointers (here, MPFR’s thread-local caches) are invisible to a collector that only scans what it knows. The system has been moving numbers *out* of the collected heap for years — and paying the copy at the border instead, as §1.6 describes. Chapter 2 is, in a precise sense, the completion of a retreat that began with this comment.

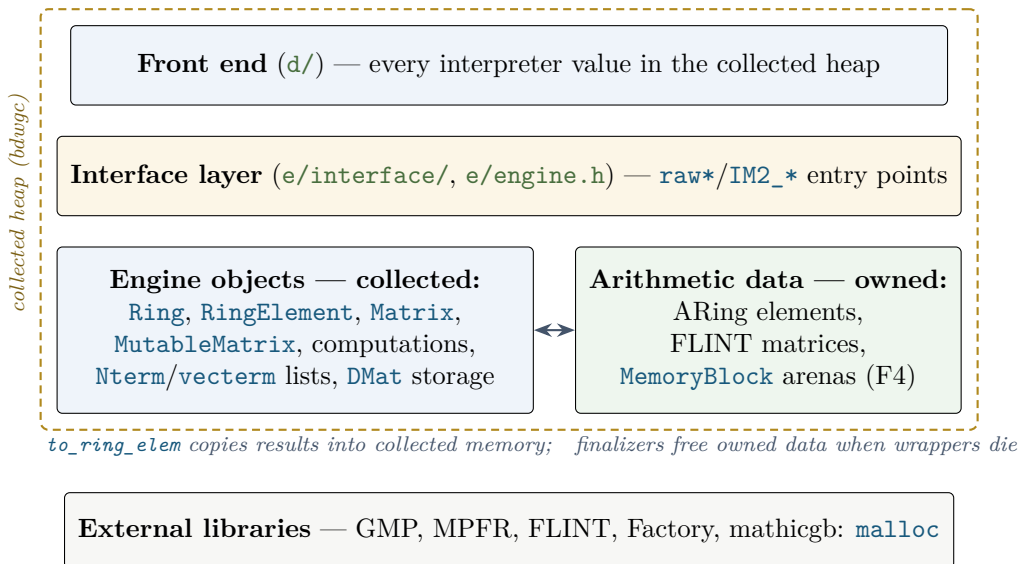


Figure 1.1: Today: the collected heap (dashed) reaches through the engine. Arithmetic data and external libraries already live outside it, with copy-in and finalizer bridges at the seam.

1.4 The life of a value: three walkthroughs

In brief

Three concrete stories — a big integer at top level, an integer computed by the engine, and a mutable matrix — show the two memory worlds cooperating in practice. The pattern is identical every time: *work happens on malloc’d data; the moment a value becomes visible to the front end, its data is copied into collected memory and the original is freed*. The system already knows exactly where its border is; it just crosses it in hand-written, case-by-case ways.

1.4.1 An integer at top level: $n = 2^{100}$

The interpreter’s big integers are GMP numbers, and the D source (d/gmp.d) opens by describing its own two-world design:

“One type is mutable, and the vector of limbs gets allocated with the standard memory allocator used by libgmp ... The other type is immutable, and the limbs are allocated with libgc by us in [a] final step after the computation.”

So when the user computes 2^{100} , the interpreter creates a *mutable* integer — a GC-visible header whose limbs GMP puts on the ordinary `malloc` heap — and lets GMP arithmetic run at full speed there. When the result is bound to `n`, `moveToZZ` performs the border crossing: allocate an atomic GC block, `memcpy` the limbs into it, repoint the header, and free the `malloc` original. The now-immutable integer sits in a `ZZcell`, one arm of the interpreter’s `Expr` union (d/parse.d —

about forty variants: lists, hash tables, strings, and one `Raw*Cell` arm per kind of engine object). From then on the collector alone decides when 2^{100} dies.

1.4.2 An integer computed by the engine

Now let the *engine* produce the number — say an entry of a matrix determinant over `ZZ`. The front end’s `ZZ` is the legacy `RingZZ` (`e/rings/ZZ.cpp`); its arithmetic allocates a GC header with `malloc`’d limbs, computes with GMP, and then — at the end of essentially every operation — calls `mpz_reallocate_limbs` (§1.6.1) so the result’s limbs live in atomic GC memory *before* the value is wrapped in a `ring_elem`. Engine integers are, in the words of a comment in `e/interface/ringelement.cpp`, “on the ‘gc’ side of the memory barrier, and ... read only” — which is why handing one to the interpreter (`IM2_RingElement_to_Integer`, reached from the D function `rawToInteger`) is a bare pointer copy, no translation needed. The interpreter wraps that pointer in a `ZZcell` and it becomes indistinguishable from a number the user typed.

Notice what this convention buys and costs. It buys safety-by-default: an engine integer can *always* be shown to the user, stored in a hash table, or captured in a closure, because it is already collector-owned. It costs a limb copy per arithmetic result — even for the vast majority of intermediate values that no user will ever see. The plan in Chapter 2 moves that copy from “every result” to “every result that actually crosses” — which is precisely how the newer ARing rings already behave (§1.6).

1.4.3 A mutable matrix, start to finish

The richest story. The user asks for a random dense matrix over $\mathbb{Z}/p$ and its rank:

1. The interpreter calls the interface layer’s mutable-matrix entry points, in `e/interface/`; the ring’s `ConcreteRing<ARingZZpFFPACK>` builds a `MutableMat<DMat<ARingZZpFFPACK>` — a garbage-collected wrapper (`MutableMatrix` is a `MutableEngineObject`, so it is GC-allocated *and* finalized) that holds the dense matrix *by value*.
2. The `DMat` inside allocates its element array — and here is the surprise flagged in §1.5: that array comes from `newarray`, i.e. *scanned GC memory*, even though its entries here are plain `doubles`. The interpreter meanwhile holds only a `RawMutableMatrixCell` — a GC cell containing the wrapper pointer — so the whole tower’s lifetime hangs on interpreter reachability.
3. The rank computation dispatches through `MatrixOps` (`mat-linalg.hpp`) straight into FFLAS-FFPACK, which grinds on the raw element array with its own `malloc`’d temporaries. During the hot loop, the collector is irrelevant — unless some *other* thread allocates and triggers a collection, in which case this thread too is paused while the collector rescans, among everything else, the matrix array it never needed to look at.
4. The user moves on; the `RawMutableMatrixCell` becomes unreachable. At some later collection, the wrapper’s finalizer runs (`e/finalize.cpp`), the `DMat` destructor clears each element and “frees” the array (a no-op, §1.3); the memory returns to the heap at the sweep after that.

1.4.4 How deep the collector’s roots go

One more vignette, not about numbers at all, to calibrate how load-bearing the collector is *above* the engine: when a background task’s handle becomes unreachable, a finalizer on the task cell *cancels the running task* (`d/pthread.d`: “cancelling inaccessible task ... still running”). Garbage collection is not just an allocator here — it is the front end’s lifetime-and-cleanup semantics, entangled with

threads, foreign functions, and user-visible behavior (`registerFinalizer` is a documented language feature). This is why Chapter 2 proposes *confining* the collector rather than removing it: above the border it is not overhead, it is the design.

1.5 A census: who allocates where

In brief

Sorting the engine’s data by memory regime shows a mostly clean pattern: *objects* (rings, matrices, computations) live in the collected heap; *arithmetic working data* (ARing elements, FLINT matrices, F4 scratch monomials, everything inside the external libraries) lives outside it. The collected side also holds the two linked-list term representations — polynomials (`Nterm`) and free-module vectors (`vecterm`) — which is where the collector and the cache both hurt, plus one genuine surprise: the storage array inside `DMat`, the dense-matrix workhorse, is itself collected memory. And exactly one coefficient ring — `CoefficientRingR`, Mike’s “wart” — computes *in* the collected heap.

1.5.1 The collected side

Table 1.1 lists the main engine residents of the collected heap. The unifying fact: these are the objects the *front end* can hold, directly or indirectly, so their lifetimes are genuinely session-shaped — the collector is doing real work here.

What	Where	Memory shape
Ring and subclasses	e/rings/ring.hpp	GC object, auto-finalized (via <code>MutableEngineObject</code>)
RingElement	e/ring-elements/	GC object: a <code>Ring*</code> plus a <code>ring_elem</code>
Matrix (immutable)	e/matrices/	GC object: <code>gc_vector</code> of columns, each a <code>vecterm</code> list
MutableMatrix	e/basic-mutable-matrices/	GC wrapper (finalized) over a dense/sparse implementation
FreeModule, MonomialIdeal, SchreyerOrder	e/free-modules/ e/monomials/	GC objects
Computation family (GB, resolutions, ...)	e/computations/	GC objects, finalized
Polynomial terms (<code>Nterm</code>)	e/rings/ringelem.hpp	hand-allocated <i>scanned</i> GC nodes, exponents inline
Vector terms (<code>vecterm</code>)	e/rings/ringelem.hpp	GC list nodes (with the in-tree TODO of §1.3.4)
Standalone monomials	e/monomials/monoid.cpp	<i>atomic</i> GC arrays (<code>newarray_atomic</code>)

Table 1.1: Principal engine residents of the collected heap today.

Two structural notes on this table. First, polynomials and matrix columns are *per-term allocations*: a polynomial with a million terms is a million separate scanned GC blocks chained by pointers. Every collection re-walks the chain; every traversal hops the heap. This one row is most of the performance story of §1.7. Second, `Nterm` nodes must be scanned (their `coeff` may point at collected data), so they cannot even use the cheaper atomic allocation — the representation forces the collector’s most expensive treatment.

1.5.2 The owned side

The modern coefficient rings — the *ARings*, in `e/basic-rings/` — are where the engine already practices what Chapter 2 preaches. An ARing is a plain C++ class satisfying an informal contract (spelled out by the base class `SimpleARing` and the worked example `DummyRing` in `aring.hpp`): a plain-data `ElementType`; GMP-style lifetime methods `init`, `init_set`, `set`, and a *static* `clear`; the arithmetic (`add`, `mult`, `invert`, ...); conversions `to_ring_elem/from_ring_elem` (§1.6); and `promote/lift/eval` for moving between rings. Each ARing also carries a tag in the (plain, non-class) `enum RingID` used for dispatch — with a sentinel `ring_old` meaning “any ring that is not one of these.” The contract even includes RAI helpers — a nested `Element` whose constructor calls `init` and whose destructor calls `clear`, and an `ElementArray` — so scoped, non-collected element lifetimes are already idiomatic. A dedicated unit-test harness exercises all of it (`e/unit-tests/ARingTest.hpp`), and the convenience wrapper Mike remembered at the meeting (“you give it the ring and an element ... then you can call `operator+`”) exists twice: as `RingElem` (`e/unit-tests/RingElem.hpp`, over the virtual `Ring` interface) and as the RAI-only `ARingElement<RT>` (`e/SLP/SLP-imp.hpp`) — a duplication §2.6 proposes to unify.

Crucially, every ARing’s *working element* lives outside the collected heap (Table 1.2) — with the single exception of §1.5.4.

Ring	Class	ElementType	Working memory
$\mathbb{Z}$	<code>ARingZZGMP</code>	<code>__mpz_struct</code>	<code>malloc</code> (GMP limbs)
$\mathbb{Z}$	<code>ARingZZ</code>	<code>fmpz</code>	<code>malloc</code> (FLINT)
$\mathbb{Q}$	<code>ARingQQGMP</code>	<code>__mpq_struct</code>	<code>malloc</code> (GMP)
$\mathbb{Q}$	<code>ARingQQFlint</code>	<code>fmpq</code>	<code>malloc</code> (FLINT)
$\mathbb{Z}/p$	<code>ARingZZp</code>	<code>int</code>	none (log tables)
$\mathbb{Z}/p$	<code>ARingZZpFFPACK</code>	<code>double</code>	none (value)
$\mathbb{Z}/p$	<code>ARingZZpFlint</code>	<code>mp_limb_t</code>	none (value)
$\text{GF}(p^n)$	<code>ARingGFM2</code>	<code>int</code>	none (log tables)
$\text{GF}(p^n)$	<code>ARingGFFlint(Big)</code>	<code>fq_zech/fq_nmod</code>	<code>malloc</code> (FLINT)
$\mathbb{R}, \mathbb{C}$ (machine)	<code>ARingRR, ARingCC</code>	<code>double(s)</code>	none (value)
$\mathbb{R}, \mathbb{C}$ (mpfr)	<code>ARingRRR, ARingCCC</code>	<code>__mpfr_struct(s)</code>	<code>malloc</code> (MPFR)
intervals	<code>ARingRRi, ARingCCi</code>	<code>__mpfi_struct(s)</code>	<code>malloc</code> (MPFI)
any legacy ring	<code>CoefficientRingR</code>	<code>ring_elem</code>	<i>collected heap</i> (§1.5.4)

Table 1.2: The coefficient-ring census (`e/basic-rings/`, plus the adapter in `e/coeffrings.hpp`). Working elements are `malloc`’d or plain values everywhere — except the last row. (`ARingTower`, experimental, mixes regimes and is omitted.)

The glue between the two ring generations is `ConcreteRing<RingType>`, in `aring-glue.hpp`: a subclass of the virtual `Ring` interface that owns its ARing by — note the type — a `std::unique_ptr`, and implements every `Ring` virtual by unwrapping `ring_elems` into ARing elements, computing, and re-wrapping. The ownership idiom of Chapter 2 is already in the tree.

Beyond the ARings, the engine’s highest-performance kernels already manage their own memory wholesale: the F4 Gröbner engine, the noncommutative F4, the newer `gb-f4/`, and the Schreyer-resolution code all stage their monomial and coefficient data in `MemoryBlock` arenas — memtailor’s `memt::Arena` under a thin wrapper (`e/MemoryBlock.hpp`), allocated with plain `operator new[]`, invisible to the collector, and released all at once when the computation ends. The external libraries (GMP, MPFR, FLINT, Factory, mathicgb) likewise `malloc` everything. In other words: *the pattern Chapter 2 proposes is already how the fastest code in the engine works*. The plan’s job is to make it

the rule rather than the exception, and to give it a policed border.

(A naming hazard for anyone reading the F4 sources: the classic F4 engine also has an `F4MemoryBlock` — which, despite the similar name, is a GC-allocated slab type. Two arena abstractions with opposite memory regimes coexist; §2.4 standardizes on the non-collected one.)

1.5.3 The dense-matrix surprise

`DMat<ARing>` (`e/basic-mutable-matrices/dmat.hpp`) is the dense matrix the linear-algebra stack computes on. Its kernels are exemplarily GC-free: `MatrixOps` dispatches by coefficient ring to FFLAS-FFPACK, FLINT, LAPACK, or Eigen, all of which treat the element array as a raw buffer. But the element array *itself* is allocated with `newarray` — scanned, collected memory — and the destructor dutifully `clears` each element and calls the no-op `freemem`. Why scanned? Because a `DMat`'s elements are sometimes *GC handles*: the adapter ring of §1.5.4 stores `ring_elems` directly in the array, and scanning is the only thing keeping those alive. So the one GC-backed coefficient ring forces the collector into every dense matrix's storage, whatever the coefficients. (The FLINT-backed specializations of `DMat` instead hold native `malloc`'d FLINT matrices — accompanied by a frank comment in `dmat-zz-flint.hpp`: “objects of this class should *not* go to the front end ... `fmpz_t`'s might be garbage collected out from under you.”) Mike's request that “all DMAT-type computations” be GC-free is thus two small steps from true, and §2.5 takes them.

1.5.4 The one exception: CoefficientRingR

The wart Mike described at the meeting — “all of a sudden, there's one A-ring type that is garbage collected, none of the other ones are” — is `CoefficientRingR` (`e/coeffrings.hpp`). It is the adapter that lets *any* legacy ring (a fraction field, a quotient ring, anything tagged `ring_old`) play the role of an `ARing`, and it does so by the simplest possible trick: its `ElementType` is `ring_elem`. Its lifetime methods say everything:

```
void init(elem &result)  const { result = R->zero(); }
void clear(elem &result) const { (void) result; }    // no-op: the collector owns it
void to_ring_elem(ring_elem &result, const elem &a)  const { result = a; }
void from_ring_elem(elem &result, const ring_elem &a) const { result = a; }
```

`clear` frees nothing because there is nothing it may free: the elements are collected objects belonging to the legacy ring. Conversions are pointer copies; the “border” is a door standing open. Any legacy ring acquires this adapter on demand (`Ring::getCoefficientRingR()`), and the F4 vector-arithmetic layer selects it for old non-prime-field coefficients — announcing itself with a stray diagnostic, “Using GC ring in VectorArithmetic.”, that ships to `stdout` today.

This also resolves the puzzle Mike voiced (“it seems to work mostly okay ... but I don't know exactly why that would be the case”): it works *because* the containers that hold these elements — `DMat`'s scanned array (§1.5.3), the GC-backed `ElementArray` this ring supplies — are visible to the collector, so reachability flows through them. The arrangement is internally consistent; its price is that it globally couples container memory to the collector. Unpicking that coupling — without breaking fraction-field coefficients — is Workstream 3 (§2.6).

1.6 The border crossings today

In brief

Data crosses between the two memory worlds at two kinds of checkpoint, both already established: *numbers* cross by having their digits copied into (atomic) collected memory — the `to_ring_elem` / `mpz_reallocate_limbs` pattern — and *owned resources* are tied to collected wrappers by *finalizers* registered in `e/finalize.cpp`. Both mechanisms work; both have accumulated inconsistencies worth cleaning up during the migration.

1.6.1 Crossing by copy: numbers

When arithmetic done outside the collected heap produces a result the front end must hold, the engine does not hand over the `malloc`'d object — it copies the number's digits (GMP's *limbs*) into atomic GC memory and releases the original. The helpers live in `e/interface/gmp-util.h`; the integer case, condensed:

```
mp_limb_t* p = (mp_limb_t*) getmem_atomic(n * sizeof(mp_limb_t));
memcpy(p, z->_mp_d, n * sizeof(mp_limb_t));
mpz_clear(z);           // free the malloc'd original
z->_mp_d = p;           // repoint at the GC-heap copy
```

with siblings for rationals, reals, intervals, and complexes (`moveTo_gmpQQ`, `moveTo_gmpRR`, `moveTo_gmpRRi`, `moveTo_gmpCC`). The copied limbs are *atomic* memory: the collector keeps them alive but never scans them — numbers cannot contain pointers, and this is the promise that makes big-number data cheap for the collector to hold.

1.6.2 Crossing by conversion: the ARing pattern

The ARings institutionalize the same copy into a named pair of methods. Inside a computation, elements are the ring's native `ElementType` (a FLINT `fmpz`, a `double`, ...) on `malloc` or in registers; `to_ring_elem` is the checkpoint where a value becomes a front-end-safe `ring_elem`. The FLINT integers show the full ritual (`e/basic-rings/aring-ZZ-flint.hpp`):

```
void to_ring_elem(ring_elem& result, const ElementType& a) const
{
    mpz_ptr b = getmemstructtype(mpz_ptr); // GC-heap mpz header
    mpz_init(b);                          // malloc'd limbs, briefly
    fmpz_get_mpz(b, &a);                   // decode the FLINT value
    mpz_reallocate_limbs(b);               // copy limbs into atomic GC memory
    result = ring_elem(b);
}
void from_ring_elem(ElementType& result, const ring_elem& a) const
{
    fmpz_set_mpz(&result, a.get_mpz());    // copy back out into FLINT
}
```

Nothing FLINT-encoded ever touches the collected heap; nothing collected is ever handed to FLINT. Each ring pays this toll only when a value actually crosses — which is the entire performance thesis of Chapter 2 in two functions. The one ring that skips the toll booth is `CoefficientRingR` (§1.5.4),

whose conversions are pointer copies because its elements never left the collected heap in the first place.

1.6.3 Crossing by finalizer: owned resources

The reverse bridge ties `malloc`-world resources to the lifetime of a collected object. `e/finalize.cpp` is the registry: its `intern_*` functions register a collector finalizer on the wrapper, and when the wrapper becomes unreachable the finalizer releases what the collector cannot see.

Class (registered by <code>intern_*</code>)	Variant	On death
<code>MonomialIdeal</code>	plain	<code>remove_MonomialIdeal()</code>
<code>GBComputation</code>	ignore-self	<code>remove_gb()</code>
<code>ResolutionComputation</code>	plain	<code>delete G</code>
<code>SchreyerOrder</code>	plain	<code>remove()</code>
<code>MutableMatrix</code>	plain	explicit destructor call

Table 1.3: The finalizer registry in `e/finalize.cpp`. (The front end registers its own finalizers separately, for task cells, mutexes, and foreign-function handles.)

Three inconsistencies in this table are worth naming, because Phase 1 of the plan (§2.9) tidies exactly these:

- **Double registration.** Every `MutableEngineObject` *already* received a finalizer at construction (from `our_gc_cleanup`, §1.3.4); the `intern_*` call then *replaces* it. The code knows this — `intern_polyring` is a documented no-op for precisely that reason — but the two-layer scheme is easy to misread and easy to get wrong when adding a class.
- **Mixed variants.** One class registers with the ignore-self variant, four with the plain one; the difference (whether a self-referential object can ever be collected) is subtle enough that it should be a policy, not an accident.
- **Mixed teardown styles.** One finalizer calls `delete`, one calls a destructor explicitly without deallocating, three call bespoke `remove` methods. With rule-of-five engine types (§2.3.1), all of these collapse into “destroy the owned object.”

One more detail for completeness: the allocator itself anticipates finalizers misbehaving — `getmem` carries a thread-local reentrancy guard with a comment noting that finalizers may legitimately allocate during collection. The plan’s finalizer policy (§2.3.2) forbids relying on that legitimacy; §1.7.4 shows why.

1.7 What the current arrangement costs

In brief

Four costs, in descending order of pain: collector time stolen from computations (Mike’s estimate has grown from “about 20%” to “sometimes a multiple-times hit”); cache-hostile linked-list representations that the collector makes both possible and expensive; memory held longer than necessary by conservative scanning in a heap that can never compact; and a hard wall that keeps FLINT’s fast integers and rationals from becoming the defaults. None of these are new observations — the tree itself is annotated with comments wishing parts of it were not collected.

1.7.1 Time

The collector charges computations in three ways. First, collections are triggered by allocation volume, and the engine’s classic representations allocate *per term*: a long polynomial multiplication churns out millions of `Nterm` nodes, each a separate collected block, so hot loops are exactly the code that summons the collector most often. Second, each collection stops every thread and re-marks the live heap — and the cost of marking is proportional to *scanned* live memory, which (§1.5) includes every term node of every polynomial a session still holds, plus every dense matrix’s element array. A computation pays rent on data it never touches. Third, Macaulay2 runs the collector in plain stop-the-world mode (§1.3); nothing is incremental, so pauses land in full.

How much does this cost end to end? Mike’s testimony at the meeting: “I used to think it was ... like a 20% hit ... I would say sometimes it’s a multiple[-]times hit ... it spends too much time in garbage collection,” together with the converse experience that removing the collector from a computation type “speeds things way up.” These are recollections, not benchmarks — which is precisely why Phase 0 of the plan (§2.9) begins by measuring collector share on a fixed suite, so every later phase can report a before/after number.

1.7.2 Cache

A linked-list polynomial is a pointer chase: each term is a separate heap block, adjacent terms need not be adjacent in memory, and every term visit risks a cache miss. As Mike put it, linked lists “just do[n]’t play well with cache.” The collected heap deepens the problem — allocation order interleaves unrelated data, and a non-moving collector can never migrate a polynomial’s terms back together. The contrast already exists in-tree: the noncommutative-algebra code (with Frank Moore) stores a polynomial as a coefficient array plus one flat integer array of all its monomials (`Polynomial<T>` in `e/Polynomial.hpp`) — the “couple [of] arrays” shape Mike sketched at the meeting. Workstream 5 (§2.8) builds on exactly that precedent.

1.7.3 Retention and fragmentation

Conservative scanning keeps anything that *looks* reachable: an unlucky integer on some stack pins a dead matrix, and everything the matrix references, until the coincidence passes (§1.2). Since the heap never compacts, long sessions fragment; and since the runtime pre-grows the heap aggressively (150 MB+ at startup, §1.3) and collects by heap-growth heuristics, a Macaulay2 process holds substantially more memory than its live data. At top level this is a fair trade for never thinking about lifetimes. Inside a computation it is pure overhead.

1.7.4 The FLINT wall

FLINT’s integer type `fmpz` is one machine word that plays exactly the kind of game a conservative collector cannot referee: small values are stored *in* the word, and large values store a *transformed* pointer to a heap structure — shifted and tagged, so the true address never appears in memory as itself. To the collector (§1.2), a large `fmpz` is a disguised pointer: invisible, hence dead, hence reclaimable out from under the computation. FLINT values are therefore banned from the collected heap — a fact the tree states plainly in the FLINT-matrix warning quoted in §1.5.3.

The consequence Mike lamented at the meeting: the FLINT integers and rationals — “the ones that are default ... should be [ZZ and QQ], but they’re not” — exist as dormant ARings. Today’s front-end `ZZ` is the legacy GMP-based `RingZZ` (not an ARing at all), today’s `QQ` is the

GMP ARing `ARingQQGMP`, and the FLINT versions are reachable only through developer functions (`rawARingZZFlint`, `rawARingQQFlint`). Making them the defaults requires nothing more — and nothing less — than a border that always copies (§2.7).

A cautionary tale

It was once tried the other way: FLINT had a mode (`FLINT_USES_GC`) for cooperating with a collector by keeping a side array of every GMP number in use, registered as a GC root. Mike, at the meeting, on how that went: the side array was modified *inside* finalization — freeing one number triggered finalizers that freed others, which rewrote the very array being traversed — “it wasn’t done in a way ... where you could alias this list. So it just didn’t work at all.” He called the resulting bug “a hellhole ... one of the worst bugs. It took me forever to find.” Without finalization the scheme limped along; with finalization it collapsed. The lesson, encoded in the finalizer policy of §2.3.2: never let finalizers mutate shared registries, and never try to make a foreign library’s `malloc` world collector-visible by bookkeeping. Copy at the border instead.

1.8 The seam is already half-built

Chapter 2 will read as less radical if this chapter’s findings are stacked up in one place. The engine already has: a named conversion ritual at the number border (`to_ring_elem` / `mpz_reallocate_limbs`, §1.6); a coefficient-ring contract with GMP-style `init/clear` lifetimes and RAII helpers, satisfied by every coefficient ring but one with working memory already outside the collector (§1.5); a glue class that owns its ring through `std::unique_ptr` (`aring-glue.hpp`); GC-free linear-algebra kernels; arena allocators in its hottest Gröbner code; a single finalizer registry (`e/finalize.cpp`); and a flattened polynomial representation in production use. It even has the opinions: the tree is dotted with comments like “`TODO: why is this garbage collected?`” (`e/rings/ringelem.hpp`) and “`TODO: make sure monoid is deconstructed and not garbage collected`” (`e/monoid.hpp`).

What it does not have is a *policed* border: `DMat` storage sits in the collected heap for the sake of one adapter ring; the legacy `ZZ` pays a limb copy on every intermediate result; finalizers are registered in two layers with three teardown styles; and nothing stops a new engine file from quietly including `newdelete.hpp`. Turning this half-built seam into an enforced architecture is what the next chapter proposes.

Chapter 2

A Plan for Confining the Collector

In brief

The plan is to *confine* the collector, not remove it. The front end keeps garbage collection forever. The engine’s computational core — ARing arithmetic, dense-matrix work, and eventually the classic polynomial/matrix types — moves to ordinary C++ ownership (`std::unique_ptr`, references, RAII), which it is already halfway to using. The only place engine code may touch collected memory is the interface layer, where results are *copied* into the collector’s heap for the front end. The migration proceeds in six phases, each independently shippable and testable, ordered so that the riskiest changes come after the mechanical ones have hardened the border. Cross-cutting rules — ownership conventions, a finalizer policy, and CI enforcement — keep the two memory worlds from bleeding into each other again.

2.1 What Mike asked for

Chapter 1 described the machine as it is. This chapter answers the specific requests Mike made at the July 2026 engine meeting (collected with timestamps in `src/mike-requests.md`). The two governing quotes:

What Mike asked for

on the goal

“If you could give us a roadmap on how to get rid of the garbage collector ...” — and immediately sharpened: “So, it’s *not* to get rid of the garbage collector. The garbage collector ... should be set up only for things in the interface directory.”

What Mike asked for

on the deliverable

A proposal document — not a giant pull request — “detailed enough for ... an overview for us to follow, and detailed enough for ... Opus or one of the lesser agents later to follow, and then we can edit the proposal.”

Table 2.1 maps every actionable request from the meeting to the part of this chapter that addresses it. Nothing Mike asked for is silently dropped; two items he himself labeled speculative are treated as optional investigations rather than plan commitments.

Request (from <code>mike-requests.md</code>)	Addressed in
Roadmap document, editable, executable in supervised steps	this chapter; §2.9
GC restricted to the interface directory / front-end objects	§2.2, §2.4
All ARing computations free of GC	§2.6
All DMat-type (dense linear algebra) computations free of GC	§2.5
Split <code>Matrix</code> into a GC'd front-end type and an engine-only owned type, with conversions both ways	§2.5
Rust-style ownership (<code>unique_ptr</code> owns; references borrow)	§2.3
Keep/clean up finalization for objects wrapping non-GC memory	§2.3, §2.4
Fix the one garbage-collected ARing (the coefficient-ring wart)	§2.6
(Adjacent) cache-friendly, non-linked-list polynomials	§2.8
(Adjacent) more FLINT rings; FLINT <code>ZZ/QQ</code> as defaults; optionally teach the collector about FLINT's encoding	§2.7

Table 2.1: Every request from the July meeting, and where this plan answers it.

2.2 The target architecture: three zones and one rule

Today the collected heap reaches all the way down through the engine (Figure 1.1, page 8). The end state confines it to the top of the picture:

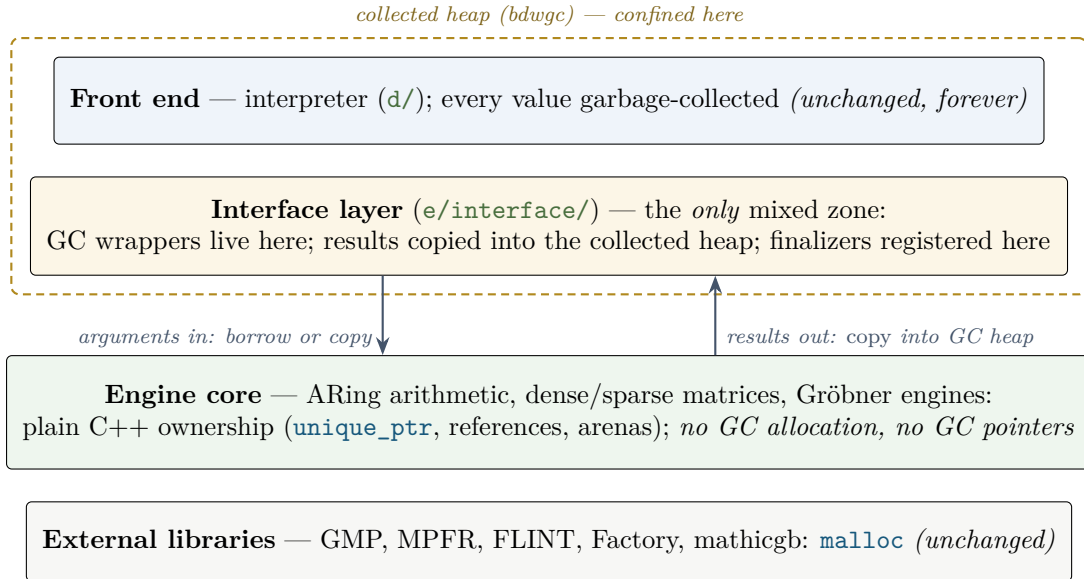


Figure 2.1: The target: the collector's reach ends at the interface layer.

Everything in this chapter serves one rule:

*Engine-core code neither allocates from the collected heap nor stores pointers into it.
Whatever crosses the border is copied, at the border, by interface-layer code.*

Three honest qualifications, so the rule is a plan and not a slogan:

- **The rule is the end state, reached subsystem by subsystem.** The linear-algebra pipeline (ARings, `DMat`, mutable matrices) gets there first because it is nearly there already;

the Gröbner engines follow; the classic term-list world (`Nterm/vecterm` polynomials and the immutable `Matrix` built from them) goes last — and parts of it may be *replaced* (§2.8) rather than converted, because migrating a linked-list representation the collector was hiding is exactly when to reconsider the representation itself.

- **“Copy at the border” is a feature, not a compromise.** Border crossings are rare relative to the arithmetic between them, and the engine *already* pays for exactly this copy where it matters — the ARing conversion functions copy number data into collected memory today (§1.6). The plan generalizes an existing, proven pattern rather than inventing a new one.
- **The front end and the external libraries do not change at all.** The collector stays; `malloc` stays; only the territory between them is renegotiated.

2.3 Ground rules for the whole migration

These conventions apply to every workstream and phase below. They are the answer to Mike’s “managed kind of like you would manage Rust objects,” made specific enough to enforce.

2.3.1 Ownership conventions

Relationship	Rust	Engine-core C++
I own it, exclusively	<code>Box<T></code> / value	value member or <code>std::unique_ptr<T></code>
I am lending it to you	<code>&T</code> / <code>&mut T</code>	<code>const T&</code> / <code>T&</code>
I am giving it to you	move	pass/return by value (move semantics)
Many owners	<code>Rc<T></code> (rare)	<code>std::shared_ptr<T></code> — <i>requires justification</i>
Raw owning pointer	forbidden	forbidden

Concretely: engine-core classes get real destructors, copy/move constructors, and `=delete` where copying is wrong — ordinary rule-of-five C++. Functions take `const&` for inputs, `&` for in-place outputs, and return results by value. A raw pointer in engine-core code always means “borrowed, non-null unless documented, never freed by the holder.” None of this is exotic; it is what the ARing element contract already does by hand with `init/clear` pairs, upgraded from convention to type system where possible.

2.3.2 Finalizer policy

Finalizers remain — but only on border-layer wrapper objects, only registered through one auditable registry (today’s `intern_*` machinery in `e/finalize.cpp`, consolidated in Phase 1), and only with bodies that are boring on purpose:

1. A finalizer may release resources owned by *its own* object (typically by `delete`-ing the engine-core object a wrapper owns — which, with rule-of-five types, frees everything underneath via ordinary destructors).
2. A finalizer must not allocate, must not touch the collected heap, must not take locks shared with allocation paths, and must not modify any global registry that other finalizers also modify.
3. Every class that registers finalizers is listed in one place, so the full set is auditable at a glance.

Rule 2 is written in blood; §1.7.4’s cautionary tale (the old `FLINT_USES_GC` side-array corrupting itself when finalizers fired during collection) is precisely what it forbids.

2.3.3 Mechanical enforcement

Conventions decay unless a machine checks them. Three cheap checks, added in Phase 1 and kept forever:

- **Include hygiene in CI.** Engine-core sources (a growing explicit list, eventually “all of `e/` outside `e/interface/`”) must not include `newdelete.hpp`, `gc-include.h`, or any `gc/*` header, and must not mention `GC_`, `getmem`, or `our_new_delete`. A grep in CI fails the build with a pointer to this document. Files not yet migrated stay off the list — so the check never blocks unrelated work, and progress is literally the list growing.
- **Sanitizer builds.** Once a subsystem is GC-free, it can finally run under AddressSanitizer and LeakSanitizer in CI (the collector currently drowns both in noise). Leaks in migrated code become test failures instead of folklore. This is a fringe benefit of the whole project worth naming: we get modern memory tooling back.
- **Border-crossing counters (debug builds).** The interface layer counts copies in and out per type. A hot loop accidentally crossing the border on every iteration shows up as a number, not a mystery slowdown.

2.3.4 Tests first, measurements always

Mike’s instinct at the meeting — put unit tests “way up above” in the checklist and work test-driven — is adopted as policy: no phase changes behavior before it has tests pinning the current behavior. The existing ARing unit-test harness (§1.5) is the model. Each phase in §2.9 names its acceptance tests. And every phase begins and ends with the same benchmark suite (fixed in Phase 0), reporting wall time and collector share, so “it speeds things way up” becomes a measured number per phase rather than an impression.

2.4 Workstream 1: harden the border

What Mike asked for	on finalization
“You tell the garbage collector, hey, when this one goes out of scope, I want you to call this other function ... typically it’s on a per-class basis, for real.” And on the state of that global picture: it is “a global piece of information that is hard to get cracked.”	

Where we are. The border exists but is unpoliced (§1.8): finalizers are registered in two layers with three teardown styles (§1.6.3); the GC surface is enumerable (`e/engine.h` plus `e/interface/*.h`) but nowhere enumerated; and nothing prevents new engine code from allocating collected memory.

The change.

1. **Make “the interface directory” literally true.** Relocate `e/finalize.cpp/.hpp` into `e/interface/`, and write the one-page description of the GC surface: which headers, which types, which `raw*` functions may traffic in collected pointers. Mike’s phrase becomes a checkable property of the tree.
2. **One finalizer mechanism.** Collapse the double registration of §1.6.3: pick a single registration site (the `intern_*` registry), a single variant (uniform ignore-self unless the Phase-1 audit finds a real cross-object cycle), and a single teardown style (destroy the owned object; with rule-of-five types that is the whole body). `our_gc_cleanup`’s constructor-time registration

either becomes that mechanism or stops being one — decided by audit, documented either way.

3. **Turn on the include-hygiene check** (§2.3.3) with its initial allowlist: the GC-free list starts with `e/basic-rings/` (minus the `CoefficientRingR` interplay), `e/MemoryBlock.hpp`, and the linear-algebra kernel headers; it grows as phases land.
4. **Small hygiene, same PR series:** remove the stray “Using GC ring” diagnostic print to `stdout` (§1.5.4); mark the fossil `stash` allocator deprecated (§1.3.4) — callers keep working, new uses are barred; rename or annotate `F4MemoryBlock` vs. `MemoryBlock` so the two arenas’ opposite regimes are visible at the call site (§1.5).

Acceptance. No user-visible change; finalizer registry uniform and listed in one file; CI check active; border counters (§2.3.3) landing numbers in debug builds.

2.5 Workstream 2: the matrix split

What Mike asked for

on matrices

“Probably the type matrix will be split into two types. One is a garbage collected one, and one’s not ... One of them is the one that’s going to talk to the front end. And one’s going to be in the engine only. Engine only, you have to free it yourself ... you would manage it like you would manage Rust objects.” And: “I don’t know exactly how to do that, so that would be the thing.”

Where we are. The split half-exists. The immutable `Matrix` (columns of `vecterm` lists, §1.5) is already, de facto, the front-end matrix; `DMat`/`SMat` are already, de facto, the engine matrices, and the GC’d `MutableMatrix` wrapper already stands between them. What is missing is exactly three things: the engine matrices’ *storage* is still collected memory (§1.5.3); the wrapper couples lifetimes by finalizer rather than by ownership; and there is no first-class conversion API — so engine code sometimes builds front-end objects mid-computation instead of converting once at the end.

The change (in order; steps 1–2 are Phase 2, steps 3–4 are Phase 4):

1. **Owned storage.** `DMat`/`SMat` allocate their element storage with an owned buffer (the `ElementArray` idiom / `std::vector`-style RAI), not `newarray`. One guard makes this safe today: a compile-time trait — call it `ring_has_collected_elements<R>` — true only for `CoefficientRingR`; a `DMat` over such a ring keeps scanned storage until Workstream 3 retires the need. Everything else (all of Table 1.2) moves out of the collected heap immediately. The FLINT specializations already hold native matrices and need nothing.
2. **Ownership-shaped wrapper.** `MutableMatrix` stays garbage-collected and finalized — it is a front-end object — but holds its engine matrix by `std::unique_ptr` instead of by value, and its finalizer body becomes “reset the pointer.” Engine algorithms never see the wrapper: they take `DMat&/SMat&`, which `mat-linalg.hpp` already does.
3. **A named conversion API**, living in the interface layer, mirroring the `ARing` convention: `toFrontEnd` producing a `Matrix`/`MutableMatrix` from an engine matrix, and `fromFrontEnd` for the reverse — copying by default, with move-adopt overloads where the element representation permits. (Names are open question 1.)
4. **Producer migration.** Computations assemble results as engine matrices and convert *once*, at the `raw*` fetch: first the linear-algebra entry points (rank, determinant, LU, solve, charpoly

— already thin wrappers over `MatrixOps`), then Gröbner/resolution result extraction. The immutable `Matrix` becomes *border native*: constructed only by interface-layer code, never inside a computation. Its internal representation (still `vecterm` lists) is then a front-end concern, free to change later without touching the engine.

Acceptance. Static: the trait gate compiles the collected-storage path only for legacy rings. Dynamic: linear-algebra unit tests run clean under AddressSanitizer; a stress test creates and drops mutable matrices in a `collectGarbage` storm; Phase-0 benchmarks (LU/rank over $\mathbb{Z}/p$ and `RR`, determinant over `ZZ`) hold or improve. API: `raw*` signatures unchanged through Phase 2; the conversion API is additive in Phase 4.

2.6 Workstream 3: ARings all the way out

What Mike asked for

on coefficient rings

“It would be really nice if all A-ring computations were not garbage collected.” On the exception: “when you do coefficient ring `R` as a simple ring ... that’s already garbage-collected memory, so eventually I would like to remove that ... It seems to work mostly okay, or all okay, but I don’t know exactly why that would be the case.”

Where we are. Every coefficient ring but one already computes outside the collected heap (Table 1.2); the contract that makes them interchangeable is real but informal; the one exception, `CoefficientRingR`, is now fully understood (§1.5.4) — including *why* it works: the containers holding its elements are scanned.

The change.

1. **Write the contract down.** The `ARing` requirements (§1.5) become a documented, linkable specification — the header-documentation effort already underway from the meeting’s first half (the A-ring headers PR) is the natural home. Includes the lifetime rule in one sentence: *working elements are yours; `ring_elems` are the collector’s; the only doors between are `to_ring_elem/from_ring_elem`.*
2. **One element wrapper.** Merge the two existing conveniences (`RingElem` with operators, unit tests only; RAII-only `ARingElement<RT>` in `SLP/`) into one blessed `ARingElement<RT>` — RAII *and* operators — promoted out of the test tree, as Mike requested at the meeting. New engine code gets scoped, leak-free elements with natural syntax.
3. **Fence the wart, then retire it.** Near term (Phase 3): `CoefficientRingR` is kept but *fenced* — the trait from Workstream 2 marks it, containers holding its elements stay scanned, it is banned from owned-storage paths, and the explanation of §1.5.4 lands as its header comment, answering the “I don’t know exactly why” on the record. Long term: it retires when the legacy rings it fronts (fraction fields first) get true `ARing` implementations — which is Workstream 4’s FLINT expansion and Workstream 5’s polynomial work, not a change we can make by editing the adapter.
4. **One infrastructure for `ZZ`.** Retire the legacy `RingZZ` in favor of `ConcreteRing<ARingZZGMP>` — same GMP semantics, same GC-side results, but expressed through the `ARing` contract like `QQ` already is (§1.7.4). This is the enabling move for Workstream 4: once `ZZ` is an `ARing` configuration, swapping GMP for FLINT is a one-line default change plus an audit, not a re-plumbing.

5. **Record the RingID decision** (plain `enum` today; `enum class` is safer and matches the meeting discussion) — open question 3, one mechanical PR whenever the group says go.

Acceptance. Contract text merged and linked from every ARing header; unified wrapper used by the unit-test suite; trait fence in place with a test that a `DMat<CoefficientRingR>` still functions; `ZZ` parity suite (identical results and hashes, legacy vs. ARing implementation) green behind a flag before the swap.

2.7 Workstream 4: FLINT by default

What Mike asked for

on FLINT

“The integers should be default, but they’re not ... same with the rationals ... because of the garbage collector, it cannot deal with encoded flint integers.” On teaching the collector: “it might be teachable at this point ... but the garbage collection code is ... not completely trivial code. I’m not sure I personally want to go in and muck around with it.” And the wish: “I would like to get more of the flint rings inside of Macaulay2.”

Where we are. The FLINT integer and rational ARings exist, dormant (§1.7.4); their border conversions already do the right thing (§1.6.2); the wall is only that the *default ZZ/QQ* wiring predates the border discipline.

The change. After Workstream 3 step 4, flipping the default is a configuration: point `globalZZ/globalQQ` at `ConcreteRing<ARingZZ>/ConcreteRing<ARingQQFlint>`. The real work is the audit around it — find every place that assumes the concrete GMP classes (casts of `globalZZ`, direct `get_mpz` access patterns, hashing, serialization), run the parity suite, and benchmark (FLINT’s small-integer encoding is precisely what makes typical coefficient arithmetic fast, so this is where “speeds things way up” should first become a public number). Ship behind a flag; flip when the group signs off. The same border then invites the rings Mike wants next — number fields, FLINT multivariates — as ordinary ARings: “a fun A-ring to try.”

Could we teach the collector instead? Answering Dima’s question from the meeting for the record: `bdwgc` does have machinery for typed objects (custom mark procedures via `GC_new_kind`) that could, in principle, decode `fmpr` words in specially tagged blocks. But it cannot help where conservatism actually bites — stacks, registers, and generic scanned memory would still hide encoded pointers — and it would put us in the business of maintaining collector internals, against the lesson of §1.7.4’s cautionary tale and against Mike’s stated appetite. Verdict: research note, not plan item (§2.11); the copy-at-border architecture makes it unnecessary.

Acceptance. Parity suite green with the flag on; benchmark deltas published; default flipped in its own, revertible PR.

2.8 Workstream 5 (adjacent): cache-friendly polynomials

What Mike asked for

on representations

“We have a number of types that are linked lists, like polynomials ... that just does not play well with cache ... polynomials are really just a couple arrays, and all the monomials are flattened out somehow ... with polynomials, it’s extremely rare that you’re doing random access into the middle.” And the hesitation, kept attached: “it locks you in a little bit ... whenever you’re hacking things, it confuses people later on.”

Where we are. The linked-list `Nterm` world and its costs are §1.5 and §1.7; the flattened counter-example already runs in production for noncommutative algebra (`Polynomial<T>`: coefficient array plus one flat, length-prefixed monomial stream, `e/Polynomial.hpp` — Frank Moore’s work), though today its arrays are `gc_vectors`.

The change (a gated pilot, runnable any time after Phase 1, not on the critical path):

1. Generalize the existing flattened representation into an engine-core `PolyArray<ARing>`: owned coefficient storage (the `ElementArray` idiom) parallel to an owned flat monomial stream; iteration is front-to-back, matching how polynomial algorithms actually read (Mike’s observation above).
2. Conversions at the border, again by the same pattern: `Nterm` $\leftrightarrow$ `PolyArray` translation functions; the linked list remains the front-end/legacy representation, so this workstream *cannot* lock the project in — retreat is deleting a type, not restoring one.
3. Pilot where translation is natural: stage Gröbner-basis input by flattening (exactly the translate-then-compute pattern Mike endorsed at the meeting for FLINT multivariates), and benchmark multiplication and reduction against the list representation, folding in the existing Stillman–Moore experiments.

Acceptance. A written benchmark verdict — adopt, adapt, or shelve — with numbers, presented at an engine meeting. No default changes without it.

2.9 The roadmap: phases, acceptance, executors

The workstreams above are *what*; this section is *when and by whom*. Each phase is one to three small pull requests, lands green, and is independently revertible; none requires a long-lived branch. Per the meeting’s intent, phases marked *agent-friendly* are written so that a supervised AI agent (“Opus or one of the lesser agents”) can execute them from this document plus the acceptance list; design-heavy steps stay with humans, with the group reviewing at the Phase 2 and Phase 4 boundaries.

Phase 0 — baseline and tooling. Fix the benchmark canon (open question 5); a starting proposal: Gröbner bases of Katsura and cyclic ideals over $\mathbb{Q}$ and $\mathbb{Z}/32003$ (default and F4 engines); a standard resolution; LU/rank/characteristic polynomial of large random `DMats` over $\mathbb{Z}/p$ (FFPACK) and `RR` (LAPACK); determinant over `ZZ`; one noncommutative F4 example; and a mutable-matrix create/drop stress under repeated `collectGarbage`. Wire each to record wall time and `GCstats` (heap size, collections, collector CPU share) before and after. Land the debug border-crossing counters. *Nothing changes behavior; an agent can do all of it.*

Phase 1 — hardened border. Workstream 1 verbatim. Mechanical, well-specified, agent-friendly;

Phase	Delivers	From	Risk	Agent-friendly
0	measurement: benchmarks, GC share, counters	—	none	yes
1	hardened border (WS 1)	0	low	yes
2	owned dense linear algebra (WS 2, steps 1–2)	1	medium	partly
3	ARing contract; wart fenced; ZZ on ARings (WS 3)	1	medium	yes
4	conversion API; producer migration (WS 2, steps 3–4)	2, 3	med–high	partly
5	FLINT defaults (WS 4)	3	medium	audit only
P	flat-polynomial pilot (WS 5)	1	research	partly

Table 2.2: The phases. “P” runs in parallel, off the critical path.

the finalizer-audit conclusions get a human review.

Phase 2 — owned dense linear algebra. Workstream 2 steps 1–2. The trait gate and storage swap are careful but bounded; the sanitizer and stress gates are the safety net. First phase expected to move a benchmark.

Phase 3 — ARing contract and the wart. Workstream 3. Mostly documentation, unification, and a flagged infrastructure swap with a parity suite — highly agent-friendly, with the [ZZ](#) flip decision reserved to the group.

Phase 4 — the split completed. Workstream 2 steps 3–4. The one phase with real API design in it: hold its design review right after Mike’s linear-algebra tour (§2.13), then migrate producers one [raw*](#) family at a time.

Phase 5 — FLINT by default. Workstream 4: agent-run audit, human-run flip, benchmark-backed victory lap — the first change a casual user might actually notice, and the direct payoff of Mike’s oldest wish in this file.

Phase P — the polynomial pilot. Workstream 5, in parallel, reporting to an engine meeting with numbers.

2.10 Risk register

2.11 What this plan deliberately does not do

Scope discipline is half the plan, so the non-goals are explicit:

- **No removal of the collector.** The front end keeps `bdwgc` permanently. Nothing here touches how the interpreter allocates.
- **No patching of `bdwgc`.** Teaching the collector about FLINT’s disguised pointers stays a research note (§2.7), not a plan item; Mike’s own verdict — “I’m not sure I personally want to go in and muck around with it” — stands.
- **No big-bang rewrite, no flag-day.** Every phase ships alone; at every commit the system builds and passes its tests.
- **No change to the external libraries.** GMP, MPFR, FLINT, Factory, and `mathicgb` keep allocating exactly as they do.
- **No new dependency and no new language.** The plan uses C++ the engine already compiles ([unique_ptr](#), moves, RAII) — Rust is a discipline here, not a toolchain.

Risk	Severity	Mitigation
A GC-heap object still holds the <i>only</i> pointer to data that moved out of the collected heap; the collector can no longer keep it alive through that pointer’s owner	crash / use-after-free	This is the central hazard of the whole project. Mitigations, layered: migrate whole subsystems (never single fields); the border copies rather than shares; wrappers own their engine object via <code>unique_ptr</code> so lifetime is explicit; sanitizer builds on migrated code; debug border-crossing counters (§2.3.3).
Border copies show up as a new cost in some workload	performance	Crossings are rare by design; Phase 0 benchmarks catch regressions per phase; for bulk data the conversion API provides move/adapt paths, not just copies (§2.5).
Finalizer-timing behavior changes (objects now die deterministically that used to die whenever the collector ran)	subtle bugs	Keep the existing <code>intern_*</code> registry as the single choke point; destruction order inside a computation becomes ordinary C++ scope order, which is <i>more</i> predictable, not less; tests pin the observable behavior.
Code outside the engine (packages, D code, external users of <code>engine.h</code>) assumes engine pointers are collector-managed	breakage	The interface layer’s types and <code>raw*</code> signatures do not change during Phases 0–3; the <code>Matrix</code> split (Phase 4) is the one deliberate API change and gets its own review round.
The classic term-list world (<code>Nterm/vecterm</code>) is too enmeshed to migrate on any near-term schedule	scope	Acknowledged up front: it stays inside the collected region until Workstream 5 replaces the representation. The plan’s wins do not depend on it — the linear-algebra and ARing paths carry most inner-loop time.
Long-lived migration branch drifts from <code>development</code>	process	Every phase is independently shippable and lands green; no phase depends on an unmerged predecessor’s branch.
AI-executed steps introduce plausible-but-wrong changes	quality	Agents execute <i>named</i> , <i>bounded</i> steps from this document only, with tests written first, small reviewable PRs, and a human (or a stricter agent) verifying against the acceptance criteria in §2.9.

Table 2.3: What can go wrong, and what keeps it from going wrong.

2.12 Open questions for the group

Decisions this document intentionally leaves to the meeting:

1. **Names.** The split matrix types need names people can live with for a decade (§2.5 proposes a pair as a starting point). Same for the conversion functions — `toFrontEnd` / `fromFrontEnd`, or match the existing `to_ring_elem` / `from_ring_elem` convention?
2. **Coordination with Doug’s dispatch work.** The `std::variant`-based promote/lift rewrite discussed at the same meeting touches the same ring-dispatch scaffolding the matrix split will cross. Which lands first, and does the variant carry the owned types or the wrappers?
3. **RingID: enum today, enum class eventually?** Raised at the meeting; cheap to do during Phase 3’s contract cleanup, but it is a wide mechanical diff — worth its own small PR.
4. **The fraction-field question.** For the one garbage-collected ARing (§2.6): fix its storage in place, or redesign how fraction fields obtain coefficient arithmetic? Mike flagged at the meeting that it “seems to work mostly okay ... but I don’t know exactly why” — Phase 3 includes finding out why before choosing.
5. **Benchmark canon.** Which computations are the official yardstick for “it speeds things way up”? Suggestions in §2.9 (Phase 0); the group should bless or amend the list.
6. **Arena strategy.** Standardize on mentailor arenas (`MemoryBlock`) for scratch memory, or move toward `std::pmr` so the standard library’s containers can participate?

2.13 The next two weeks

Matching what was agreed at the meeting — a proposal now, edits before code:

1. **Circulate this document** (Zulip + the engine meeting), with `src/mike-requests.md` alongside so the requests can be checked against the plan. Mike marks up Chapter 2; disagreements become edits, not debates in PR comments.
2. **Bless the benchmark canon** (open question 5) and land Phase 0’s measurement tooling — it is zero-risk and useful regardless of every later decision.
3. **Settle open questions 1–3** (names, Doug coordination, `RingID`) — each is a five-minute decision that unblocks a phase.
4. **Schedule the engine tour.** Mike offered a walkthrough of the linear-algebra organization at a meeting when attendance is good; the matrix-split design review (§2.5) is best held right after it.

If the group signs off on the shape of this plan, Phase 0 can begin immediately — it changes no behavior, and everything after it is a sequence of small, testable, individually revertible steps toward the architecture of Figure 2.1.

Appendix A

Glossary

Terms of art used in this document, in plain language.

Arena (also: region, pool)

A big block of memory from which many small allocations are carved by bumping a pointer. Freeing is all-or-nothing: you throw away the whole arena when the computation finishes. Extremely fast, and immune to fragmentation. The engine's `MemoryBlock` is an arena.

ARing (“algebra ring”)

The engine's newer family of ring implementations (`e/basic-rings/aring-*.hpp`). Each ARing is a concrete C++ class with a plain-data element type and a fixed set of required methods (`init`, `clear`, `add`, ...) — a *contract* in the informal sense. ARings do their arithmetic outside the garbage-collected heap (with one exception, `CoefficientRingR`) and convert at explicit crossing points.

Atomic allocation

In Boehm-GC jargon, memory that the collector will *not* scan for pointers, because the caller promises it contains none (strings, arrays of numbers). Allocated with `GC_MALLOC_ATOMIC`. Unrelated to atomic operations in threading.

Boehm GC (bdwgc)

The Boehm–Demers–Weiser conservative garbage collector, a C library that replaces `malloc`: you allocate and simply stop freeing; the collector periodically finds unreachable memory and reclaims it.

Conservative collection

A collection strategy that does not know the layout of your data. Anything in memory that *looks like* a pointer into the collector's heap is treated as one. Consequence: the collector can

never move objects, and integers that happen to look like addresses keep dead objects alive (*false retention*).

DMat

The engine's dense-matrix template (`e/basic-mutable-matrices/dmat.hpp`), parameterized by an ARing: a flat array of ring elements plus dimensions. The workhorse of the linear-algebra code paths.

Engine

The C++ half of Macaulay2 (`Macaulay2/e/`): rings, matrices, polynomials, Gröbner bases, resolutions. Sits below the interpreter, above the external arithmetic libraries.

Finalizer

A function you register with the garbage collector to be called when a particular object is about to be reclaimed. Macaulay2 uses finalizers to let garbage-collected wrapper objects release resources that the collector cannot see (`malloc`'d memory inside FLINT, Factory, `mathicgb`, ...).

Front end

The Macaulay2 interpreter: the language the user types, implemented in the D language under `M2/Macaulay2/d/` and translated to C. All interpreter values live in the garbage-collected heap.

GC root

A starting point for the collector's reachability search: stacks, registers, globals, and anything explicitly registered. Everything reachable from a root is kept; everything else is garbage.

Interface layer

The C-linkage functions (mostly in

`e/interface/`) through which the interpreter calls the engine. In Mike’s plan this layer becomes the *only* place where engine code touches garbage-collected memory.

Interned object

An engine object registered with the collector so a cleanup action runs when it dies; see the `intern_*` functions in `e/finalize.cpp`. Used for Gröbner bases, resolutions, monomial ideals, Schreyer orders, and mutable matrices.

Limbs (GMP)

GMP stores a big integer as a small header plus an array of word-sized digits called *limbs*. Where the limbs live (GC heap vs. `malloc` heap) decides which memory regime owns the number.

Mark-and-sweep

The collector’s algorithm: *mark* everything reachable from the roots, then *sweep* (reclaim) everything unmarked. The program is paused during at least part of this.

Ownership (Rust-style)

The discipline where every object has exactly one owner responsible for freeing it; others hold non-owning references. In C++ this is spelled `std::unique_ptr` (owner), `&/const &` (borrower), and “never a raw owning pointer.”

RAII

“Resource Acquisition Is Initialization” — the

C++ idiom where a destructor releases whatever the constructor acquired, so cleanup is automatic and exception-safe. The engine-only types in Chapter 2 are all RAII types.

ring_elem

The engine’s universal element handle (`e/rings/ringelem.hpp`): one machine word that is either a small immediate value or a pointer to garbage-collected data, depending on the ring. The old virtual-**Ring** interface traffics entirely in `ring_elems`.

Smoke test

A quick “does it even turn on?” test — create the object, do one operation, confirm nothing crashes. (Term borrowed from electronics: power it up and see if smoke comes out.)

Stop-the-world pause

The interval during which the collector halts every program thread to scan memory. Frequent pauses inside inner loops are one of the two ways the collector costs us time (the other is cache pollution from scanning).

vecterm / Nterm

The linked-list nodes from which the classic engine builds free-module vectors (**vecterm**) and polynomials (**Nterm**): a coefficient, a monomial, and a `next` pointer, allocated node by node in the GC heap.